

Jobs to Be Done

DCSA Unified Application Access and Integration Prototype (DCSA-UAL)

Field	Value
Product Name	DCSA Unified Application Access and Integration Prototype
Project Acronym	DCSA-UAL
Document Type	Jobs to Be Done (JTBD)
Status	Draft v1.0
Date	2026-09-14
Related Personas	project_specs/PERSONAS-DCSA-UAL.md (PER-01 ... PER-04)
Related PRD	project_specs/PRD-DCSA-UAL.md (§5 Feature Requirements; §7 Success Metrics)
Related Charter	.planning/PROJECT.md

DEMO — SYNTHETIC DATA ONLY. Every job, measure, and criterion below is written to be **observable in a live demonstration** on synthetic data. No real DCSA personnel, applicants, cases, or systems are represented.

How to Read This Document

A job is **what the user is trying to get done**, stated independently of the product that helps them do it. None of the jobs below is "use a portal." The jobs are things like *make a defensible trust determination without losing the thread across four systems and clear the blocking issue on a case in one sitting so the investigation keeps moving*.

The friction these jobs exist to remove is specific and it is the whole reason for the product: today a DCSA user with **one** piece of work to finish signs into **several** separate applications — eApp, IEP, PVQ, PDT, Investigation Management — re-authenticating at every hop, hand-carrying identifiers between systems in a scratch document, and losing the thread of what they were doing at every boundary. The systems were organized around applications instead of around the work.

Each job carries:

- **Job Statement** — "When ... I want to ... so I can ..."
- **Job Context / Trigger** — the real situation in which the job arises
- **Current Workarounds** — how the job gets done today across the fragmented estate
- **Desired Outcomes** — written in canonical outcome form: *direction + metric + object + context*
- **Hiring Criteria** — what would make this persona adopt the unified layer for this job
- **Firing Criteria** — what would make them abandon it and go back to the separate applications
- **Success Measure** — one quantifiable, demo-observable outcome
- **Related Features** — PRD feature IDs (F0-F19)

Priorities inherit from the PRD: **P0** = the demonstration fails without it; **P1** = required for a credible complete prototype; **P2** = valuable, first to be cut.

JTBD Summary

ID	Persona	Job Statement (abbreviated)	Priority
----	---------	-----------------------------	----------

JTBD-01.1	PER-01 Investigator	When a case is blocked by an issue raised against a questionnaire answer, I want to clear it in one sitting, so the investigation keeps moving. (Flagship — F7)	P0
JTBD-01.2	PER-01 Investigator	When I start my day with 40+ open cases, I want one trustworthy answer to what is due first, so I prioritize by evidence rather than by whoever called last.	P0
JTBD-01.3	PER-01 Investigator	When I record a finding, I want a record that names exactly what changed and where, so my work survives scrutiny months later.	P0
JTBD-01.4	PER-01 Investigator	When a source system is degraded, I want to be told what I cannot currently see, so I can keep working without acting on a false picture.	P1
JTBD-02.1	PER-02 Adjudicator	When a determination lands in my queue, I want the whole cross-system record assembled for me, so the reading is the work instead of the assembly.	P0
JTBD-02.2	PER-02 Adjudicator	When I render a determination, I want it recorded as a durable correlated chain, so it is defensible on appeal years later.	P0
JTBD-02.3	PER-02 Adjudicator	When timeliness is measured on me, I want aging and blocked determinations surfaced continuously, so I manage the clock instead of discovering it.	P1
JTBD-02.4	PER-02 Adjudicator	When I see no issues on a subject, I want certainty that means "none exist," so I never decide on silently incomplete data.	P1
JTBD-03.1	PER-03 Applicant	When weeks pass with no word, I want a plain-language answer to where I stand, so I stop guessing and stop calling my security officer.	P0
JTBD-03.2	PER-03 Applicant	When something is required of me, I want to know exactly what and do it immediately, so a deadline never passes because I did not know.	P0
JTBD-03.3	PER-03 Applicant	When the government needs to tell me something, I want it where I already check, so I do not miss it in a lost email.	P1
JTBD-03.4	PER-03 Applicant	When asked whether I sent something in, I want to answer with evidence — and know only I can see my file.	P1
JTBD-04.1	PER-04 Administrator	When the next application must join the platform, I want to onboard it as configuration in one sitting, so integration stops being an engineering project. (F12)	P1
JTBD-04.2	PER-04 Administrator	When I start my day, I want to know the state of every connected application before a user reports a problem. (F11/F16)	P1
JTBD-04.3	PER-04 Administrator	When a user reports a failure, I want to classify it from one place in minutes, so triage stops being manual log correlation.	P1
JTBD-04.4	PER-04 Administrator	When asked "who did what, to what, when," I want one correlated chain, so the answer is a record rather than an assertion.	P1

Coverage note. PER-01 carries the flagship job (JTBD-01.1 → F7) — the single artifact that proves the product thesis. PER-04 carries the extensibility job (JTBD-04.1 → F12) and the operability job (JTBD-04.2 → F11/F16), which are the two

architectural claims a reviewer will test. PER-02 and PER-03 prove the same unified layer composes and enforces differently per role; their jobs are deliberately *not* variants of PER-01's.

PER-01: Marcus Vale — Jobs

Background investigator, DCSA Personnel Vetting. 38–45 active cases. Measured on cases closed on time and on the defensibility of what he recorded. Works across Investigation Management, eApp, and PVQ to advance a single case.

JTBD-01.1: Clear a Blocking Issue on a Case in One Sitting

Job Statement: When a case I own is held up by an issue raised against something the subject answered on their questionnaire, I want to read the answer and dispose of the issue in one continuous sitting, so I can keep the investigation moving before the case ages.

Job Context / Trigger: This is the flagship job and the most frequent case-advancing action Marcus performs — several times weekly. He is working an eApp case from his queue, reaches a section where PVQ has raised an issue item against a specific answer (e.g., Section 13A employment history), and cannot advance the case until that issue is dispositioned. The two halves of one decision — the answer and the issue — live in two systems that do not share a session, a search, or a reference format. He is frequently interrupted mid-task and must be able to return 40 minutes later and still be where he left off.

Current Workarounds:

- Authenticates separately to eApp to read the answer, then separately to PVQ to find the issue — the second session has usually timed out by the time he needs it.
- Keeps a scratch document open holding the subject identifier, case number, and issue reference, because neither system accepts a reference from the other. A transposed digit pasted into the wrong system is a recurring, real error class.
- Searches PVQ manually for the issue by the reference he wrote down, then reconstructs in his own head what he was trying to accomplish — the cognitive reset costs more than the clicks.
- After acting, checks each system separately to confirm the change actually landed in both, because nothing tells him whether the pair is consistent.

Desired Outcomes:

- Minimize the number of authentication events required to complete a single case-advancing action when that action spans more than one mission system. (*Target: exactly one per session.*)
- Minimize the number of identifiers re-typed or pasted by hand when moving between a questionnaire answer and the issue raised against it. (*Target: zero.*)
- Minimize the time to reach the related issue from the case that is blocked by it, when the relationship already exists in the data.
- Increase the likelihood that both affected systems reflect a completed action, when the action legitimately changes state in two places.
- Minimize the likelihood of being told an action succeeded when only part of it did, when a coordinated write partially fails.

Hiring Criteria:

- The issue raised against a questionnaire answer is visible **on the case itself**, with the relationship explained in words ("Issue raised against Section 13A employment history"), sourced live rather than hard-coded (F6 related-items panel, F7).
- Opening that issue happens **inside the same shell** — same header, same breadcrumb trail carrying the case context, no interstitial login, no new tab, no visual discontinuity (F3, F7).

- Resolution is a real, persisted action: disposition plus narrative, validated, with the confirmation naming **which systems changed** and what each now says, read back from each spoke independently (F6, F7).
- If the second write fails, he is shown an explicit partial-completion state naming the exact systems affected, with a retry path — never a silent success (F7).
- The whole path is completable **keyboard-only**, and a long resolution narrative is never lost to a validation failure (F14).
- Returning to the queue afterwards restores his prior filters, sort, and page (F6).

Firing Criteria:

- Any credential prompt, second browser tab, or "you are now leaving this application" interstitial mid-workflow — that is the exact pain he came here to escape.
- Having to re-key a case, subject, or issue identifier at any step.
- A success message that turns out to be false in one of the two systems. One instance of this destroys trust in the entire product for him.
- Losing a typed narrative to a validation error or a silent session timeout.

Success Measure: The investigator completes eApp case → related PVQ issue → resolution → dual-system confirmation in **under three minutes, with exactly one authentication event in the audit log, zero manual identifier re-entry, and both spokes independently returning the updated state via their own APIs** (SM-01, SM-02, SM-03, SM-04).

Related Features: F7 (flagship), F1, F2, F3, F6, F9, F10, F13, F14 **Priority: P0 — highest priority job in the product**

JTBD-01.2: Decide What to Work Next Across My Whole Caseload

Job Statement: When I sit down with 40-odd open cases and a day that will be interrupted a dozen times, I want one trustworthy answer to what is assigned to me and what is due first, so I can spend my attention on the highest-consequence item rather than on reconstructing my own workload.

Job Context / Trigger: Daily, multiple times — first thing Monday after a weekend of accumulation, and again after every interruption. Marcus is a throughput worker measured on on-time closure. The consequence of bad prioritization is not inefficiency, it is an aged case and a supervisor conversation.

Current Workarounds:

- Reads IM for what cases he owns, PVQ for open issue items, and eApp for questionnaires awaiting review — three lists, three sort orders, none prioritized against the others.
- Prioritizes by habit and by whoever called him most recently, because no system answers "what first."
- Discovers a newly raised issue on one of his cases accidentally — usually when it is already late, or when his supervisor asks.
- Rebuilds his filters and scroll position from scratch every time he returns from an interruption.

Desired Outcomes:

- Minimize the number of separate systems consulted to establish what is assigned to me, in the context of a caseload spanning four mission applications.
- Minimize the time to identify the next item to work, when items originate from different systems with different native priority semantics.
- Increase the proportion of caseload changes that are announced rather than discovered, when something new is raised against a case I own.
- Minimize the effort to re-establish working context, when returning to the queue after an interruption of 10-60 minutes.

Hiring Criteria:

- One queue aggregating all five spokes, each row carrying **visible source-system attribution**, filterable and sortable by due date, priority, status, type, and source (F5).
- A default view that already answers his question — "assigned to me, due date ascending" — without configuration (F5).
- Alerts that name the exception (newly raised PVQ issue on a case he owns, overdue item, blocked case) and link **directly** into the item that produced it (F15, F4).
- The next item he should work is reachable from the dashboard in **two clicks or fewer**.
- Returning from an item restores filters, sort, and page; sortable headers announce sort state and filter changes announce a result count (F6, F14).

Firing Criteria:

- A queue that shows fewer sources than he knows he has work in — he will go back to checking each system to be sure.
- Priority or due-date semantics that differ invisibly between sources, so sorting produces an order he cannot explain.
- Losing filter and sort context on every return trip, forcing him to rebuild the view a dozen times a day.

Success Measure: The investigator's unified queue renders correctly attributed items from **at least four of five spokes**, and he opens the correct next item from the dashboard in **two clicks or fewer** (SM-14, SM-05).

Related Features: F5, F4, F15, F3, F2, F17 **Priority:** P0

JTBD-01.3: Leave a Record That Holds Up Months Later

Job Statement: When I record a finding or a disposition on a case, I want the record to state plainly what changed, in which system, and when, so that an adjudicator reading it months from now — or an inspector reading it years from now — can trust what I did.

Job Context / Trigger: Daily. Marcus's output is consumed out of context by PER-02 and potentially re-examined long after he has forgotten the case. His defensibility is a measured dimension of his performance. Before acting on a contested item he also needs to read what already happened to it — currently a manual merge of activity histories from separate systems.

Current Workarounds:

- Pulls activity history from each system separately and lines the timestamps up by hand; nobody can see a cross-system action as one event because no system recorded it as one.
- Writes context into the free-text narrative that the system should have captured structurally, because he knows the reader will not have the other three screens.
- Confirms his own writes by re-opening each system, since confirmations name no system and quantify nothing.

Desired Outcomes:

- Minimize the time to establish the full history of a work item, when that history spans more than one mission system.
- Increase the proportion of recorded actions whose confirmation names the affected system and the resulting state, in the context of cross-system writes.
- Minimize the effort to reconstruct a cross-system action as a single narrative, when a case is contested or reviewed after the fact.

Hiring Criteria:

- Every state-changing action writes an audit record **before** success is returned — an action that cannot be audited does not complete (F13, NFR-06).

- Item activity history merges the spoke's own history with hub audit records into one chronology showing actor, action, timestamp, and originating system (F6, F13).
- A cross-system action is retrievable as **one correlated chain** under a single correlation ID, not four disconnected rows (F13).
- Confirmation states name exactly what changed and where; failure states distinguish "not permitted" from "source unavailable" from "invalid input," each with a recovery path (F6).
- He can see his own recent activity on his dashboard, with links back to the affected items (F4, F13).

Firing Criteria:

- History that shows hub events but not what the spoke recorded, or vice versa — a partial record is worse than none because he will trust it.
- Any confirmation that says "Saved" without naming the system.
- Discovering that an action completed without an audit entry.

Success Measure: 100% of the investigator's state-changing actions produce exactly one audit record, and the flagship cross-system action renders as a **single correlated chain** in the audit viewer (SM-19, SM-20).

Related Features: F13, F6, F10, F2, F1 **Priority:** P0

JTBD-01.4: Keep Working — and Keep Trusting the Screen — When a System Is Down

Job Statement: When one of the systems behind my queue is unhealthy, I want to be told which one and what I am therefore not seeing, so I can keep working on everything else without acting on a picture I wrongly believe is complete.

Job Context / Trigger: Roughly 40% of Marcus's week is on VPN from an employer site, a hotel, or a car. Connectivity is intermittent and spokes are occasionally degraded. Today the symptom of an unhealthy back-end is an empty list, not a warning — so he cannot distinguish "no items" from "the system could not be reached," and an empty screen becomes untrustworthy by default. If the page simply stalls, he assumes the whole system is broken and gives up.

Current Workarounds:

- Treats an empty list as suspicious and re-checks the source system directly to confirm it is genuinely empty.
- Calls the administrator or a colleague to ask whether "it's just me."
- Stops working and waits, because he has no way to know which part of his picture is missing.

Desired Outcomes:

- Minimize the time to distinguish an empty result from an unavailable source, in the context of an aggregated view spanning five systems.
- Maximize the proportion of the work queue that remains usable and actionable, when a single source system is unavailable.
- Minimize the number of unrecoverable states (error pages, blank screens) encountered, when any spoke degrades.
- Minimize the user effort required to resume full function, when a degraded system recovers.

Hiring Criteria:

- A prominent, accessible degraded-system warning that **names the application and quantifies the gap** — "Investigation Management is unavailable — 12 items are not shown" (F16, NFR-10).
- The other four sources render fully and remain actionable; one slow spoke never stalls the whole page (F16, F5).
- Actions targeting an unavailable spoke are pre-emptively disabled with an explanation rather than allowed to fail mid-submission (F16).
- When the spoke returns, the warning clears and data reappears **without a reload and without re-authenticating** (F16).

- Warnings are delivered through live regions **without stealing focus** — he may be mid-form when a refresh completes (F14).

Firing Criteria:

- An error page, a blank screen, or a stack trace anywhere in the product.
- An empty list that was actually an outage. Once he learns the screen can lie by omission, he re-checks every source system by hand forever and the product has lost its only value.
- A warning that steals focus out of a half-written narrative.

Success Measure: With a spoke forced offline via the administrator's failure-injection control, the investigator's queue renders the remaining sources plus a **specific named warning, with no error page anywhere in the application**; restoring the spoke clears the warning **without reload or re-authentication** (SM-15, SM-16, SM-17).

Related Features: F16, F5, F4, F14, F9, F11 **Priority:** P1

PER-02: Dana Okonkwo — Jobs

Adjudicator, Consolidated Adjudication Services. 15–25 cases pending determination; 25–90 minutes each. Reads across eApp, PVQ, PDT, and IM to make one decision that must be defensible to a reviewing authority, an appeal, and an inspector general.

JTBD-02.1: Assemble the Whole Picture on a Subject Before Deciding

Job Statement: When a subject reaches me for a determination, I want the complete record — what they submitted, what was questioned, how each question was disposed, and what tier the position required — in front of me in one reading path, so I can spend my time judging the evidence instead of collecting it.

Job Context / Trigger: Several times daily. One determination requires four systems: the eApp questionnaire as submitted, every PVQ issue raised against it *and how each was dispositioned*, the PDT designation that set the investigation tier (adjudicating against the wrong tier is a finding against her), and the IM case record showing investigative completeness. She holds an enormous amount of detail in working memory across a 45-minute read; anything that forces her to re-establish that context is expensive.

Current Workarounds:

- Signs into four systems separately for one subject and assembles the picture herself, on paper or in a side document. **The assembly is the work, and it is unbillable.**
- Re-authenticates several times per case because session lifetimes are mismatched — the PVQ session has expired by the time she has finished reading in eApp.
- Correlates by identifier, by hand: the issue references a case, the case references a subject, the designation references a position, and nothing resolves those references for her.
- Reads a resolution narrative in PVQ with no way to see, in the same place, the questionnaire answer it was raised against — the relationship exists in the data and not on her screen.

Desired Outcomes:

- Minimize the number of authentication events required to review one subject's complete record, when that record spans four mission systems.
- Minimize the time to locate all information bearing on a single subject, when that information lives in more than one mission system.
- Increase the proportion of on-screen fields whose originating system is identifiable, in the context of a record assembled from multiple sources.

- Minimize the loss of reading context, when traversing from a questionnaire answer to the issue raised against it and back.

Hiring Criteria:

- A related-items panel that resolves cross-system relationships inline — the eApp case links to its PVQ issues, its PDT designation, and its IM assignment — so traversal replaces four searches (F6, F7).
- The investigator's **disposition and resolution narrative** are visible, not merely the fact that an issue existed and closed. The quality of her determination depends on the quality of that record (F6, F13).
- **Source-system attribution on 100% of work-item rows and detail panels**, present in the accessible name and not conveyed by a colored badge alone (F5, F6, F14). She will ask where a field came from; "the system" is not an answer.
- One session across the whole reading path — no re-authentication at any boundary (F1).
- Correct heading hierarchy, landmarks, and programmatic grouping so a dense multi-source record is navigable by heading and by keyboard (F14, F3).

Firing Criteria:

- Any field she cannot attribute to a source system. Unattributed data is, to her, unusable data.
- A related-items panel that is decorative — links that do not resolve, or relationships that are demonstrably stale or hard-coded.
- Being re-prompted for credentials part-way through a 45-minute read.

Success Measure: The adjudicator reaches **every artifact needed for one determination — questionnaire, issue dispositions, designation/tier, case status — without leaving the unified shell and without re-authenticating**, with source attribution present on 100% of rows and detail panels (SM-02, SM-04).

Related Features: F6, F5, F7, F1, F3, F14, F17 **Priority:** P0

JTBD-02.2: Render a Determination That Is Defensible Years Later

Job Statement: When I decide a subject's eligibility, I want the decision recorded with who I was, what I could see, and what I did at that moment, so the determination stands up to a reviewing authority, an appeal, or an inspection long after I have forgotten the case.

Job Context / Trigger: Several times daily, and the single most consequential action in the product. Her write surface is deliberately narrow — record a determination, return for additional investigation, request clarification, annotate — and each of those is a government decision about an individual. She must also be confident the system is enforcing the boundary between her role and the investigator's: she may legitimately *view* a PVQ issue item she may not *resolve*.

Current Workarounds:

- Records the determination in one system and documents its basis in a side note, because no single record ties her decision to the material she saw.
- Reconstructs "what happened on this case, in what order" only when a case is contested — meaning she usually decides without it.
- Relies on convention rather than enforcement to stay inside her role, because each system interprets her permissions on its own terms and she is never confident she has seen everything she is entitled to see.

Desired Outcomes:

- Increase the proportion of determinations accompanied by a durable, correlated record naming the actor and their attributes at the time of action.
- Minimize the effort to reconstruct the sequence of events on a case, when the case is contested after the determination.

- Increase confidence that the actions offered to me are exactly the actions I am entitled to perform, in the context of a shared work item also handled by other roles.

Hiring Criteria:

- Available actions are **computed server-side per item and per principal**, and re-authorized at execution time — viewing an item never implies the right to act on it (F2, F6).
- The audit record captures timestamp, actor identity, **roles and attributes held at the time**, action type, target system and resource, outcome, correlation ID, and before/after summary (F13).
- Audit storage is append-only with sequence numbering and hash chaining, and **no application path exists** to modify or delete a record — for anyone, including administrators (F13, NFR-07).
- Determination forms are long and high-consequence: error summary with focus management and in-page links to offending fields, so a validation failure never costs her the narrative (F14).
- Opening the same PVQ issue item as the investigator presents a **demonstrably different, server-computed action set** (F2 — and the clearest live RBAC demonstration in the product).

Firing Criteria:

- Any action offered to her that the server then refuses — a UI that advertises what she cannot do makes her distrust every other control.
- A determination that completes without an audit record, or an audit record she can see a path to edit.
- Losing a long determination narrative to a form error.

Success Measure: The adjudicator and the investigator open **the same work item and are presented with different, server-computed action sets**; her determination writes **exactly one audit record before the success response returns**, and a direct API call to the investigator-only resolve action is denied server-side and audited (SM-18, SM-19).

Related Features: F2, F6, F13, F10, F14 **Priority:** P0

JTBD-02.3: Manage the Clock Before the Clock Manages Me

Job Statement: When my determinations are measured on timeliness, I want the aging and blocked cases in my queue surfaced to me continuously, so I manage the deadline with evidence instead of discovering it in an end-of-month report.

Job Context / Trigger: Daily, with an acute end-of-month push. A case sitting in her queue because she is waiting on a clarification from an investigator is a case she needs surfaced, not buried. Today aging is invisible until it becomes a metric someone else reports on.

Current Workarounds:

- Learns a case is late from a management report rather than from her workspace.
- Builds her own aging view by hand, or does not build it and absorbs the risk.
- Tracks "blocked pending investigator response" in memory or a side list, because no system models the wait.

Desired Outcomes:

- Minimize the time to identify which pending determinations are aging or blocked, in the context of a queue assembled from multiple source systems.
- Increase the proportion of timeliness exceptions that are surfaced proactively rather than reported retrospectively.
- Minimize the manual effort required to distinguish "waiting on me" from "waiting on someone else."

Hiring Criteria:

- Sorting and filtering on due date and age **across sources**, with clear and consistent date semantics regardless of which spoke supplied the row (F5).

- An alerts panel computing overdue, approaching-due, and blocked/stalled cases server-side, each linking directly to the item that produced it (F15, F4).
- Requesting clarification from an investigator marks the case blocked and tracks it as such until his response arrives (F6, F15).
- Status conveyed by **text and icon, never color alone** — "resolved," "open," "returned," "overdue" distinguishable in grayscale (NFR-02, F14).

Firing Criteria:

- An aging view she has to build herself — that is the workaround she already has.
- Date semantics that differ silently between sources, making a sorted list wrong in a way she cannot see.
- Alert counts that do not reconcile with what is actually in her queue.

Success Measure: The adjudicator's dashboard and queue answer "**what is aging and what is blocked**" without her constructing a report, with every alert resolving to a real, populated destination (SM-05, SM-06).

Related Features: F4, F5, F15, F6, F14 **Priority:** P1

JTBD-02.4: Never Decide on Data That Is Quietly Missing

Job Statement: When I see no issues raised against a subject, I want to know for certain that means none exist, so I never render a determination on a record that was incomplete because a system was quietly unreachable.

Job Context / Trigger: Structural to her role and asymmetric in cost. An empty issue list in PVQ looks identical whether the subject genuinely has no issues or the service is degraded. Adjudicating on incomplete data because a spoke was down is a career-relevant error, and it is invisible at the moment it is made.

Current Workarounds:

- Treats empty results with suspicion and re-checks the source system directly — when she remembers, and when she has time.
- Delays determinations she is unsure about, adding to the aging problem she is separately measured on.
- Accepts the risk, because there is no signal available to her either way.

Desired Outcomes:

- Minimize the probability of rendering a determination on incomplete data, when one or more source systems are degraded.
- Minimize the time to distinguish a genuine absence of records from an unavailable source, in the context of adjudicative review.
- Increase the proportion of incomplete views that state explicitly what is missing and why.

Hiring Criteria:

- When PVQ is unavailable, she sees a **specific named warning stating that issue data is missing** — never an empty list that reads as "no issues" (F16, NFR-10).
- Every list, table, and widget has a **designed empty state** that is visibly different from a degraded state (F16).
- Actions that depend on an unavailable source are disabled with an explanation rather than permitted to fail (F16).
- Degraded warnings are announced via live regions without stealing focus mid-read (F14).

Firing Criteria:

- One instance of an empty list that turns out to have been an outage. The trust loss is permanent and total for this persona.
- A generic "something went wrong" that does not name the system or the gap.

Success Measure: With PVQ forced offline, the adjudicator's subject record shows a **specific named warning that issue data is unavailable — and never an empty issue list** (SM-15, NFR-10).

Related Features: F16, F6, F5, F14, F9 **Priority:** P1

PER-03: Renée Ashford — Jobs

The individual being vetted — a private-sector systems engineer at a cleared defense contractor undergoing an initial Tier 3 investigation. Her start date on billable work depends on this process completing. She has no mental model of DCSA's internal system boundaries and should never need one.

JTBD-03.1: Find Out Where I Actually Stand

Job Statement: When weeks have gone by with no word about my clearance and my start date depends on it, I want a plain-language answer to where I am in the process, so I can stop guessing and stop calling my security officer to ask.

Job Context / Trigger: Roughly 70% of her sessions exist for this question alone. She checks in short anxious bursts — fifteen minutes at lunch, twenty minutes after the kids are in bed — frequently **on a phone**, and often after eleven days away, having forgotten everything about the interface. Long silence with no visible progress leads her to assume something is wrong. She does not know, and must not need to know, that the answer is assembled from IEP and eApp.

Current Workarounds:

- Checks the questionnaire in one system and the status in another, with different terminology for the same state, and cannot tell whether the two screens describe the same thing.
- Reads internal process jargon as if it were a status. "Pending SOI transmittal" is not a status; it is a barrier.
- Calls her facility security officer, generating work for three other people to answer a question the interface should have answered.
- Resets a forgotten password on one of her two logins — each reset costs her an evening.

Desired Outcomes:

- Minimize the time to determine current standing in the vetting process, when that standing is derived from more than one government system.
- Minimize the number of credentials required to see one process end to end, in the context of a single application for eligibility.
- Minimize the number of unexplained internal terms encountered, when reading one's own status as a first-time applicant.
- Minimize the number of support contacts initiated to answer a question the interface could answer.

Hiring Criteria:

- One progress narrative in plain language, sourced from IEP and eApp and presented as **one process, not two system statuses** (F4, F5).
- **Zero internal system names, tier codes, or state abbreviations** surfaced without plain-language explanation — she should never need to learn that eApp and IEP are different things (F4, F14).
- Usable on a phone: readable at **320px width with no horizontal scroll**, at 200% zoom, with adequate touch targets (NFR-16, F14).
- Progress conveyed by **text and structure**, not by a color-coded graphic alone (NFR-02).
- Her dashboard is visibly and substantively different from the three mission-role dashboards — same product, different composition (F4).

Firing Criteria:

- Any acronym or internal state name she has to look up. She will conclude the system is not for her and call her security officer instead.
- Scrolling past a fold of jargon before reaching her status, especially on a phone.
- A second login to see the other half of her own process.

Success Measure: The applicant can answer "**where am I in this process**" within **30 seconds of signing in, on a phone, without scrolling past a fold of jargon** — and the screen contains **zero unexplained internal system names or state codes** (SM-05, NFR-16).

Related Features: F4, F5, F1, F3, F14, F17 **Priority:** P0

JTBD-03.2: Know Exactly What I Owe Next — and Do It Now

Job Statement: When the government needs something from me, I want to see precisely what it is and complete it in the same sitting, so a deadline never passes because nobody told me plainly what was required.

Job Context / Trigger: Episodic but critical. Her sessions are short and very frequently interrupted — a child, a doorbell — and she abandons and resumes. Today she typically learns she owes something when a deadline has nearly passed, through a forwarded email from her facility security officer rather than from the systems themselves. She does not know which system owns the step and should not have to.

Current Workarounds:

- Waits for her security officer to forward an email, then tries to work out which system the request refers to.
- Re-reads an email chain to determine whether a request was already answered.
- Abandons a partly completed form when interrupted and re-enters everything on her next attempt, because a timeout discarded her progress.

Desired Outcomes:

- Minimize the time to identify every outstanding obligation, when obligations are issued by more than one government system.
- Minimize the number of steps between learning that something is required and completing it, in the context of a short, interrupted session.
- Minimize the volume of entered information lost to session expiry, when sessions are short and frequently interrupted.
- Increase the proportion of requests discharged before their due date.

Hiring Criteria:

- A **short, ordered, unambiguous list** of outstanding tasks with due dates — not a systems inventory (F4, F5).
- **Every task links directly to the action that discharges it.** No dead ends and no "contact your security officer" as a primary path (F6, NFR-14).
- Forms with associated labels, hint text, required-field indication that is not color-only, inline errors, and an error summary with focus management — she is disclosing personal detail she may find uncomfortable, and a confusing validation error compounds that (F14).
- Session timeout warns her and offers an accessible re-authentication path that **does not discard entered data** (F0).
- Submitting produces a confirmation that names **what was received and what happens next** (F6).

Firing Criteria:

- A task with no way to complete it from where she found it.
- Losing a partly completed form to a silent timeout. She will not start it a third time in the same week.
- A confirmation that says only "Submitted," leaving her unsure whether anything actually happened.

Success Measure: 100% of the applicant's outstanding tasks link directly to the action that discharges them, and a submission produces a confirmation naming what was received and what happens next, with the submission then visible in her own history (SM-06, NFR-14).

Related Features: F5, F6, F4, F0, F14 **Priority:** P0

JTBD-03.3: Hear About Changes Where I Already Look

Job Statement: When something changes in my case or the government needs to tell me something, I want the notice waiting where I already check my status, so I do not miss it in an email I never received or already lost.

Job Context / Trigger: Episodic, high value. She may not return for eleven days. Notices today arrive by email — sometimes forwarded second-hand by a security officer — and get lost. She has no way to tell whether she has already read and acted on something.

Current Workarounds:

- Searches her personal email for anything from the contractor's security office.
- Relies on her facility security officer as a human notification service.
- Re-reads old messages to determine what is still outstanding, with no read/unread state to guide her.

Desired Outcomes:

- Minimize the likelihood of missing a required notice, when notices are delivered outside the place status is checked.
- Minimize the time to determine which notices are new since the last visit, in the context of visits weeks apart.
- Minimize reliance on a third party to relay official communications about one's own case.

Hiring Criteria:

- Notices are rendered **in the same place she checks status**, retained, with read/unread state and an accessible mark-as-read action (F15, F4).
- Each notice links directly to whatever it requires of her, preserving context (F15).
- New notices are announced via live regions **without stealing focus** (F14, F15).
- Announcements are dismissible per user and never obscure the demo banner (F15, F3).

Firing Criteria:

- Finding out about a requirement from her security officer after the fact — that means the product did not do the one thing it promised.
- A notice list that does not distinguish what she has already handled.

Success Measure: A notice issued to the applicant is **visible in-app on her next sign-in with unread state, links directly to the required action, and is announced without stealing focus** (F15 acceptance, SM-06).

Related Features: F15, F4, F6, F14, F3 **Priority:** P1

JTBD-03.4: Prove What I Submitted — and Know Nobody Else Can See It

Job Statement: When my security officer asks whether I sent something in, I want to answer with evidence rather than memory — and at the same time be confident that the most sensitive information I have ever disclosed is visible only to me.

Job Context / Trigger: Occasional but disproportionately reassuring. The record she is filling contains her most sensitive personal information, and she has no visible assurance that the system is showing her only her own record — or that it is showing her record only to her. This job is also the product's **headline zero-trust demonstration**: an applicant is the strictest access scope in the system and the most important negative test.

Current Workarounds:

- Searches email to prove what she submitted and when.
- Keeps her own copies of forms, because she cannot retrieve them from the system afterward.
- Assumes, without assurance, that her record is appropriately restricted.

Desired Outcomes:

- Minimize the time to produce evidence of a prior submission, when asked by an employer's security officer.
- Increase confidence that only one's own records are accessible, in the context of a system serving many applicants.
- Minimize the probability that another party can retrieve an applicant's record through any access path.

Hiring Criteria:

- Her own submission and activity history, self-scoped, showing what she sent and when, with links back to the item (F6, F13 self-scoped).
- **Every read is resource-level entitlement-checked against her subject identity** — route-level checks alone are a defect (F2).
- A direct API call to another subject's record is denied **server-side**, returns a **non-enumerable error that does not reveal whether the record exists**, and the denial is written to the audit trail (F2, F13).
- She is never shown investigative content, investigator notes, PVQ issue items raised against her answers, PDT designations, or IM case records — that asymmetry is a deliberate boundary, not an omission (F2).

Firing Criteria:

- Any glimpse of another individual's information, however incidental.
- An error message that reveals whether another record exists.
- A history that omits something she knows she submitted.

Success Measure: An authenticated applicant calling an investigator-only endpoint — **or a legitimate endpoint with another subject's resource ID — is denied server-side with a consistent non-enumerable error, and the denial appears in the audit trail**, demonstrable live via curl (SM-18, F2 acceptance signal).

Related Features: F2, F13, F6, F10, F19 **Priority:** P1

PER-04: Priya Raghunathan — Jobs

Platform administrator for the unified layer itself, in the Integrated Enterprise program office. She does not investigate, adjudicate, or apply — she is responsible for the thing all three depend on. She is also the persona who proves the product's two architectural claims: that it is extensible (F12) and that it is operable (F11, F16).

JTBD-04.1: Bring the Next Application onto the Platform Without an Engineering Project

Job Statement: When a new mission application needs to join the platform, I want to onboard it as a configuration change I perform myself in one sitting, so integration stops being a bespoke engineering project with its own budget, deployment, and set of assumptions.

Job Context / Trigger: Rare in production, **central to the demonstration**, and strategically her most important job. Every new integration today is custom code, a new deployment, and a new set of assumptions, because there is no repeatable integration contract and no registry. That cost is the reason the ecosystem stays fragmented — which means this job is the one that determines whether the platform's premise holds beyond the five systems it launched with.

Current Workarounds:

- Raises an engineering effort: requirements, custom integration code, a release, a regression cycle. Weeks to months, per application.
- Negotiates a bespoke contract with each system's team, because no common interface exists to point at.
- Cannot try an integration before committing to it — there is no way to test a candidate endpoint short of building against it.

Desired Outcomes:

- Minimize the elapsed time to make a new application's work visible to its users, when that application already exposes a conformant interface.
- Minimize the number of code changes, deployments, and service restarts required to add an application, in the context of a running platform.
- Minimize the time to discover what a candidate application can do, when the application can describe its own capabilities.
- Minimize the number of teams that must be involved to complete an onboarding.

Hiring Criteria:

- A guided multi-step UI flow: identity → connection → capabilities → role access → review and confirm, with accessible step indication and per-step validation (F12, F14).
- **Capability auto-discovery via `describe()`** pre-populating work-item types and supported actions from the application's own declaration, shown for confirmation (F12, F8).
- A **live connection test** during registration whose result is shown — and announced via a live region — before submission is allowed (F12, F14).
- On submit, the application appears **immediately** in admin inventory, health monitoring, role-scoped navigation, and the unified work queue — **with no hub restart** (F12, F8, NFR-11).
- Registration, edit, and de-registration are all fully audited, naming the administrator, the application, and the configuration (F12, F13).
- Removing an application from the registry removes it cleanly from navigation, queue, and console with no code change and no errors (F8).

Firing Criteria:

- Any step that requires a developer, a pull request, a redeploy, or a restart — that is the status quo she is trying to escape.
- A registration flow that only works for the one application shipped with the demo, i.e. hard-coded extensibility.
- An unreachable endpoint discovered *after* submission rather than caught by the connection test.

Success Measure: The administrator registers the **sixth application live through the UI in under five minutes with zero code changes and zero restarts**, and the investigator sees its work items in his unified queue **without signing out** (SM-11, SM-12, NFR-11).

Related Features: F12, F8, F11, F13, F14, F9, F18 **Priority:** P1 (*the extensibility proof — non-negotiable for the demonstration narrative*)

JTBD-04.2: Know the Integration Surface Is Healthy Before Anyone Complains

Job Statement: When I start my day, I want to see at a glance which connected applications are healthy, degraded, or unavailable, so I learn about a failing integration from my console rather than from an investigator calling to say his queue is empty.

Job Context / Trigger: Daily at start of day, and on demand whenever anything is reported. Her rhythm is monitoring punctuated by incident. Today, the first signal that an integration is failing is a user complaint — by which point the incident has been running for an hour and has an audience. Health is checked per system, by different teams, with different definitions of "up," and there is no inventory that answers "what is attached to this platform right now."

Current Workarounds:

- Checks each system's own status page or endpoint separately, if one exists.
- Maintains an informal list of what is connected, which drifts from reality.
- Waits for the phone to ring, and treats the complaint itself as the monitoring signal.

Desired Outcomes:

- Minimize the time to determine the health of every connected application, in the context of a platform spanning six or more independently operated systems.
- Increase the proportion of integration failures detected by monitoring rather than by user report.
- Minimize the delay between an adapter beginning to fail and that failure being visible in the console.
- Minimize the effort to confirm recovery, when a previously failing application returns to service.

Hiring Criteria:

- A **connected-applications inventory** read directly from the registry — display name, identifier, adapter type, endpoint, exposed work-item types, supported actions, enabled state, registration date (F11, F8).
- **Per-application health** with current status, last successful check, response latency, and recent check history from a background health monitor (F11, F16).
- A manual **"test connection"** action performing a live check on demand from the application detail view (F11).
- Health status conveyed by **text label and/or icon, never a red/green dot alone** — a conformance failure and an operational hazard (NFR-02, F14).
- Circuit-breaker behavior visible in its effects: the hub stops hammering a failing spoke and restores automatically on recovery via half-open probing (F16).

Firing Criteria:

- A health view that reports "up" while users are experiencing failures — a dashboard that lies is worse than no dashboard.
- Health state that only refreshes on page load, so the console is stale at the moment she is trusting it.
- Discovering a registered application missing from the inventory, or an inventory entry for something no longer connected.

Success Measure: Inducing an adapter failure produces a **correctly attributed entry in the integration error log and a degraded/unavailable health state within one health-check interval**, and restoring the spoke returns it to healthy **without any manual intervention** (F11 acceptance signal, SM-17).

Related Features: F11, F16, F8, F14, F9, F10 **Priority:** P1

JTBD-04.3: Classify a Reported Failure in Minutes, from One Place

Job Statement: When a user reports that something is broken, I want to determine within minutes whether it is a spoke outage, an adapter misconfiguration, an authorization denial, or user error, so the answer to "what happened" stops being an afternoon of manual log correlation.

Job Context / Trigger: Bursty and urgent. Long monitoring lulls, then a high-pressure window where **time-to-answer is the metric**. Today a single user-reported failure means pulling logs from the hub and from each candidate system and lining up timestamps by hand, with nothing tying the records to one another — because nothing ever issued a correlation ID.

Current Workarounds:

- Exports logs from the hub and every candidate system, then merges timestamps manually in a spreadsheet.
- Asks the user to reproduce the failure while she watches, because there is no recorded trace to consult.
- Guesses at the failure class from the symptom and escalates to whichever team seems most likely.

Desired Outcomes:

- Minimize the time to classify a reported failure by cause, in the context of a request that traverses a hub and multiple spoke systems.
- Minimize the number of separate log sources that must be consulted to trace one user action end to end.
- Increase the proportion of failures whose full trace is retrievable by a single identifier.
- Minimize the time to decide and execute a containment action, when one application is determined to be at fault.

Hiring Criteria:

- A filterable **integration error log** with timestamp, application, operation, error class, correlation ID, and affected principal, **linked to the corresponding audit entries** (F11, F13).
- **Correlation IDs propagated end to end** from browser through hub to spoke, into both audit and error logs, and surfaced in user-facing error states so a user can hand her the ID (NFR-15, F16, F10).
- Correlation IDs presented as **selectable, copyable text** — not images, not truncated cells without an accessible full value. She pastes them into tickets constantly (F14).
- The ability to **disable an application from the console**, audited, and immediately reflected in user-facing navigation and queue (F11).
- Published API documentation she can read and endpoints she can call directly with curl, because she will verify rather than assume (F10).

Firing Criteria:

- A trace that stops at the hub boundary and does not reach the adapter call.
- Error log entries without a correlation ID, or correlation IDs that do not actually correlate across logs.
- A console action that changes behavior but leaves no audit record.

Success Measure: Given a correlation ID from a user-facing error state, the administrator retrieves **the complete chain of hub and adapter operations for that action in a single filtered query**, and disabling the implicated application is reflected in user navigation and queue immediately and is itself audited (SM-20, F11 acceptance signal).

Related Features: F11, F13, F16, F10, F14, F8 **Priority:** P1

JTBD-04.4: Answer "Who Did What, to What, When" as a Record Rather Than an Assertion

Job Statement: When someone asks me to account for a cross-system action after an incident or an inquiry, I want to produce one correlated chain of record, so my answer is evidence rather than a reconstruction I am asking them to believe.

Job Context / Trigger: As needed, and always under scrutiny. She is the person asked after any incident. Today, auditability is per-system: reconstructing a cross-system action means correlating multiple logs by hand, and the result is an assertion rather than a record. She also needs to rehearse resilience deliberately — inducing controlled failure so degraded behavior is verified before it is discovered in production — and to communicate to users without an email blast.

Current Workarounds:

- Assembles a narrative from several partial logs and caveats it heavily.
- Cannot demonstrate that a record has not been altered, because nothing is sequenced or chained.
- Has no way to induce a controlled failure, so resilience behavior stays untested until it is production.
- Emails whole user populations to communicate a platform change, with no way to target by role or confirm relevance.

Desired Outcomes:

- Minimize the time to produce a complete account of a cross-system user action, in the context of an inquiry or incident review.

- Increase the proportion of platform actions — including administrative ones — that are attributable to a named actor with the attributes they held at the time.
- Minimize the probability that an audit record can be altered or deleted through any application path.
- Minimize the time to verify degraded-system behavior, in the context of a platform whose failure modes cannot otherwise be rehearsed.

Hiring Criteria:

- An **audit viewer** filterable by actor, role, action type, target system, resource, outcome, and date range, with a **chain view** showing all records sharing a correlation ID (F13).
- Coverage beyond mutations: authentication events, **authorization denials**, registration changes, and adapter failures are all recorded (F13).
- **Administrators are not exempt** — her own console actions are authorized server-side and appear in the audit trail attributed to her; and administrative privilege over the platform is explicitly **not** privilege over mission content (F11, F2).
- Append-only storage with sequence numbering and hash chaining, with the **absence of mutation paths verified by test** (F13, NFR-07, F19).
- **Failure-injection controls** (administrator-only, demo-scoped) to force any spoke into unavailable, slow, or erroring state and then restore it (F16).
- Role-targeted **system announcements** with severity and effective/expiry dates, rendered on the right dashboards and dismissible per user (F11, F15).

Firing Criteria:

- Any gap in the chain — a write that produced no audit record, or a denial that was never recorded.
- Any path, for any role, that can edit or delete an audit record.
- Failure injection that leaves the platform in a state a restore does not fully clear.

Success Measure: Any cross-system user action is retrievable as a **single correlated chain** in the audit viewer; **the administrator's own console actions appear in that trail attributed to her**; and an attempt to read mission work-item content or mutate an audit record is denied and itself audited (SM-20, SM-19, F11 acceptance signal).

Related Features: F13, F11, F16, F2, F15, F19, F18 **Priority:** P1

Outcome-to-Feature Traceability

JTBD ID	Job	Features	Expected Outcome
JTBD-01.1	Clear a blocking issue in one sitting	F7 , F1, F2, F3, F6, F9, F10, F13, F14	A case-advancing action spanning eApp and PVQ completes in one continuous sitting with one authentication, zero identifier re-entry, and both spokes independently confirming the new state
JTBD-01.2	Decide what to work next	F5, F4, F15, F3, F2, F17	One prioritized queue spanning at least four of five spokes replaces three per-system lists; the next action is two clicks from sign-in
JTBD-01.3	Leave a defensible record	F13, F6, F10, F2, F1	Every state change is auditable, attributed, and retrievable as one correlated cross-system chain months later
JTBD-01.4	Keep working when a system is down	F16, F5, F4, F14, F9, F11	Degradation is named and quantified; the rest of the queue stays usable; recovery is automatic and requires no reload or re-authentication

JTBD-02.1	Assemble the whole picture on a subject	F6, F5, F7, F1, F3, F14, F17	Questionnaire, issue dispositions, designation/tier, and case status are reachable in one reading path, with source attribution on every field
JTBD-02.2	Render a defensible determination	F2, F6, F13, F10, F14	Determination actions are server-computed per principal and per item, and produce a durable audit record naming actor and attributes at the time
JTBD-02.3	Manage the clock	F4, F5, F15, F6, F14	Aging and blocked determinations are surfaced continuously and link directly to the item, without the adjudicator building a report
JTBD-02.4	Never decide on missing data	F16, F6, F5, F14, F9	An unavailable source is always distinguishable from a genuine absence of records, by name and by quantity
JTBD-03.1	Find out where I stand	F4, F5, F1, F3, F14, F17	A single plain-language progress narrative sourced from IEP and eApp, readable on a phone in under 30 seconds, with no unexplained internal terms
JTBD-03.2	Know what I owe and do it now	F5, F6, F4, F0, F14	Every outstanding obligation is listed with a due date and a working direct path to discharge it, and progress survives interruption
JTBD-03.3	Hear about changes where I look	F15, F4, F6, F14, F3	Notices are delivered and retained in the same place status is checked, with read state and a direct link to the required action
JTBD-03.4	Prove submissions, trust the boundary	F2, F13, F6, F10, F19	Self-scoped submission history is retrievable, and every cross-subject or cross-role access attempt is denied server-side, non-enumerably, and audited
JTBD-04.1	Onboard the next application	F12 , F8, F11, F13, F14, F9, F18	A sixth application is registered live through the UI in under five minutes with zero code changes and zero restarts, and becomes immediately visible to users
JTBD-04.2	Know the integration surface is healthy	F11 , F16 , F8, F14, F9, F10	Health, latency, and check history for every registered application are visible before a user reports a problem, with failures logged within one check interval
JTBD-04.3	Classify a reported failure fast	F11, F13, F16, F10, F14, F8	A user-supplied correlation ID resolves to the complete hub-and-adapter trace in one query, with containment available from the console
JTBD-04.4	Account for who did what	F13, F11, F16, F2, F15, F19, F18	Cross-system actions — including the administrator's own — are retrievable as one immutable, tamper-evident correlated chain, and degradation can be rehearsed on demand

Feature Coverage Check

Every PRD feature is claimed by at least one job. No job exists without a feature to deliver it.

Feature	Serving Jobs	Feature	Serving Jobs
F0	01.1, 03.2	F10	01.1, 01.3, 02.2, 03.4, 04.2, 04.3

F1	01.1, 01.3, 02.1, 03.1	F11	01.4, 04.1, 04.2, 04.3, 04.4
F2	01.1, 01.2, 01.3, 02.2, 03.4, 04.4	F12	04.1
F3	01.1, 01.2, 02.1, 03.1, 03.3	F13	01.1, 01.3, 02.1, 02.2, 03.4, 04.1, 04.3, 04.4
F4	01.2, 01.4, 02.3, 03.1, 03.2, 03.3	F14	all sixteen jobs
F5	01.2, 01.4, 02.1, 02.3, 03.1, 03.2	F15	01.2, 02.3, 03.3, 04.4
F6	01.1, 01.3, 02.1-02.4, 03.2, 03.3, 03.4	F16	01.4, 02.4, 04.2, 04.3, 04.4
F7	01.1, 02.1	F17	01.2, 02.1, 03.1
F8	04.1, 04.2, 04.3	F18	04.1, 04.4
F9	01.1, 01.4, 02.4, 04.1, 04.2	F19	03.4, 04.4

NaC Preview

Candidate Natural Acceptance Criteria derived from each job's success measure. These are the seeds that STORY-MAP and the verification plan will refine — each is written so a reviewer can observe it pass or fail during a live demonstration on synthetic data.

JTBD ID	Outcome	Candidate NaC
JTBD-01.1	Cross-system work completes in one sitting	Given an investigator signed in once, when he opens an eApp case, follows its related PVQ issue, records a disposition and narrative, and submits, then both eApp and PVQ return the updated state via their own independent APIs, the audit log contains exactly one authentication event for the session, and no identifier was re-entered at any step.
JTBD-01.1	Partial failure is never reported as success	Given the second of two coordinated writes fails, when the action is submitted, then the investigator is shown a partial-completion state naming exactly which system changed and which did not , with a retry path — and no success confirmation is displayed.
JTBD-01.1	The whole path is keyboard-operable	Given a keyboard-only user, when the flagship workflow is driven start to finish without a pointing device, then every step is reachable and operable, focus is placed predictably on each cross-application traversal, and the page title changes descriptively.
JTBD-01.2	One trustworthy answer to "what first"	Given an investigator's default queue view, when the work queue loads, then it contains correctly attributed items from at least four of five spokes , is sorted "assigned to me, due date ascending," and announces its result count when a filter changes.
JTBD-01.2	Changes are announced, not discovered	Given a PVQ issue is newly raised against a case the investigator owns, when he next views his dashboard, then an alert names the exception and links directly to the work item that produced it.
JTBD-01.3	Actions are auditable and correlated	Given any state-changing action, when it completes, then exactly one audit record was written before the success response returned, and the full cross-

		system action is retrievable as a single correlated chain in the audit viewer.
JTBD-01.4	Degradation is named, never blank	Given Investigation Management is forced offline, when the investigator loads his queue, then the remaining four sources render fully, a warning names IM and quantifies the missing items , and no error page or blank screen appears anywhere in the application.
JTBD-01.4	Recovery is automatic	Given a previously offline spoke is restored, when the next health check succeeds, then the degraded warning clears and data reappears without the user reloading or re-authenticating .
JTBD-02.1	The record assembles itself	Given an adjudicator reviewing one subject, when she traverses questionnaire → issue dispositions → designation/tier → case status, then she never leaves the unified shell, is never re-prompted for credentials , and each field displays its source-system attribution in its accessible name .
JTBD-02.2	Authorization is action-level and server-computed	Given the adjudicator and the investigator open the same PVQ issue work item , when the available actions are rendered, then the two action sets differ, are computed server-side, and a direct API call by the adjudicator to the investigator-only resolve action is denied and audited .
JTBD-02.3	Aging is surfaced, not reported	Given pending determinations of varying age, when the adjudicator loads her dashboard, then aging and blocked items are identified without her constructing a report, and every alert resolves to a real, populated destination.
JTBD-02.4	Absence and outage are distinguishable	Given PVQ is unavailable, when the adjudicator opens a subject record, then a specific named warning states that issue data is unavailable and no empty issue list is presented that could be read as "no issues exist."
JTBD-03.1	Status is plain, fast, and phone-ready	Given an applicant signing in on a 320px-wide viewport, when the dashboard loads, then her overall status is readable within 30 seconds without horizontal scrolling , and the screen contains zero internal system names, tier codes, or state abbreviations without plain-language explanation.
JTBD-03.2	Every obligation has a working path	Given the applicant has outstanding tasks, when the task list renders, then 100% of tasks link directly to the action that discharges them (no dead ends, no "contact your security officer" as the primary path), and submitting produces a confirmation naming what was received and what happens next.
JTBD-03.2	Interrupted work survives	Given an applicant mid-form when her session approaches expiry, when the timeout warning appears, then she is offered an accessible re-authentication path that does not discard entered data .
JTBD-03.3	Notices live where status lives	Given a notice is issued to the applicant, when she next signs in, then it appears in-app with unread state , links directly to the required action, and is announced via a live region without stealing focus .
JTBD-03.4	The privacy boundary is enforced server-side	Given an authenticated applicant, when she calls an investigator-only endpoint or a legitimate endpoint with another subject's resource ID directly, then the request is denied server-side , returns a consistent non-enumerable error that does not reveal whether the resource exists, and the denial is written to the audit trail .
JTBD-04.1	Onboarding is configuration, not	Given the demo sixth application is running but unregistered, when the administrator completes the registration flow in the UI, then registration finishes

	code	in under five minutes with zero code changes and zero restarts , and the application appears immediately in inventory, health monitoring, role navigation, and the work queue.
JTBD-04.1	New work reaches users without a session break	Given the sixth application has just been registered, when the investigator refreshes his queue without signing out , then its work items appear, correctly attributed to the new source.
JTBD-04.1	De-registration is equally clean	Given a registered application, when it is removed from the registry, then it disappears cleanly from navigation, queue, and console with no code change and no errors , and the change is audited.
JTBD-04.2	Health is known before it is reported	Given an induced adapter failure, when one health-check interval elapses, then the application's health state changes to degraded/unavailable and a correctly attributed entry appears in the integration error log , with status conveyed by text and/or icon rather than color alone.
JTBD-04.3	One identifier resolves the whole trace	Given a correlation ID taken from a user-facing error state, when the administrator filters the error log and audit viewer by that ID, then the complete chain of hub and adapter operations for that action is returned in a single query, with the ID present as selectable, copyable text.
JTBD-04.3	Containment is immediate and audited	Given an application implicated in a failure, when the administrator disables it from the console, then it is removed from user-facing navigation and queue immediately and the change is written to the audit trail.
JTBD-04.4	Administrators are not exempt	Given the administrator performs a console action, when she queries the audit trail, then her own action appears attributed to her ; and an attempt to read mission work-item content, or to mutate an audit record through any application path, is denied and itself audited .
JTBD-04.4	Resilience is rehearsable	Given the failure-injection controls, when the administrator forces a spoke unavailable and then restores it, then the specific degraded warning appears across the product, no error page occurs anywhere, and the warning clears automatically on recovery.

Open Questions Carried Forward

#	Question	Affects	Disposition
JQ-01	Does the applicant ever need visibility that an issue was raised against one of her answers, or is the mediated "request for information" task the only acceptable surface?	JTBD-03.2, JTBD-03.3	Assumption: mediated only. PVQ issue items are a mission-user surface and are deliberately not exposed to the applicant (PER-03 Access Scope). Revisit only with DCSA policy input.
JQ-02	Is "return for additional investigation" a distinct job for the adjudicator or a variant action within JTBD-02.2?	JTBD-02.2, JTBD-02.3	Modeled as an action within JTBD-02.2, with its blocked-state consequence carried by JTBD-02.3. Split if STORY-MAP finds the acceptance criteria diverging.
JQ-03	Which additional work-item types does the demo sixth application ("Continuous Vetting	JTBD-04.1,	Assumption: extends JTBD-01.2 only. The onboarding pattern is the point; new mission

	Service") expose, and do they create a new investigator job or only extend JTBD-01.2?	JTBD-01.2	capability is explicitly out of scope (PRD §9).
JQ-04	Does the investigator require a job around <i>initiating</i> a clarification request to the applicant, distinct from recording a finding?	JTBD-01.3	Folded into JTBD-01.3 as an action. Promote to its own job if the mediated applicant touchpoint (PER-01 ↔ PER-03) grows acceptance criteria of its own.

Downstream Use

Consumer	What it takes from this document
STORY-MAP	Jobs become the backbone activities; NaC Preview rows become the seed acceptance criteria per story
User Journeys	Job Context/Trigger and Current Workarounds define the as-is journey; Desired Outcomes define the to-be
FRD / Functional Requirements	Hiring Criteria are the functional obligations; Firing Criteria are the negative and edge cases that must be handled
Verification / Test Plan	Success Measures map to PRD success metrics (SM-01 ... SM-22) and to the F19 automated suite
Demo Script (F18)	JTBD-01.1 is the primary script; JTBD-02.2, JTBD-03.4, JTBD-04.1, and JTBD-01.4 are the four secondary scripts